# Autonomous 3D Quadrotor UAV Controller via Deep Reinforcement Learning with Hardware-in-the-Loop Implementation on AMD Kria KR260

Author Name (NIM: XXXXXXXX)

Academic Year 2026/2027 — Institut Teknologi Bandung

**Abstract**

This report documents the design, simulation, training, and deployment of a Deep Reinforcement Learning (DRL) 3D flight controller for an unmanned aerial vehicle (UAV). To overcome common sim-to-real brittleness and actuator chattering observed in traditional reinforcement learning setups, a hierarchical architecture is introduced. The outer-loop policy runs at 50 Hz and commands collective vertical thrust and 3D body rates, while an inner-loop rate PID controller operates at 500 Hz to handle rapid gyroscopic and motor dynamics. The environment is simulated with high fidelity in MuJoCo, and training is executed using multi-process Proximal Policy Optimization (PPO). The trained policy achieves sub-5 cm steady-state hover precision under nominal conditions, with seamless export paths to the AMD Kria KR260 FPGA/SoC platform.

# 1 Introduction

## 1.1 Background and Motivation

Autonomous flight control for unmanned aerial vehicles (UAVs) has traditionally relied on cascaded Proportional-Integral-Derivative (PID) or Linear Quadratic Regulator (LQR) architectures. While effective near equilibrium, these linear approaches degrade when subjected to aerodynamic nonlinearities, unmodeled cross-coupling, and persistent external disturbances such as wind gusts.

Deep Reinforcement Learning (DRL) provides a data-driven alternative capable of directly mapping multi-sensor observations into optimal control commands. However, direct end-to-end motor PWM control often results in high-frequency bang-bang switching (actuator jitter) and severe sim-to-real gaps. This project addresses these challenges through action low-pass filtering, smooth reward shaping, and domain randomization.

## 1.2 Problem Statement and Objectives

The objective of this work is to: 1. Formulate a 6-DoF Quadrotor dynamics model and simulation environment in MuJoCo. 2. Develop a jitter-free DRL policy capable of 3D waypoint regulation and precision hover. 3. Quantify performance under nominal conditions and stochastic aerodynamic disturbances. 4. Establish a Hardware-in-the-Loop (HIL) deployment architecture on the AMD Kria KR260 FPGA platform.

# 2 System Modeling and Simulation

## 2.1 Quadrotor Dynamics

The quadrotor is modeled as a 0.8 kg rigid body in an X-configuration with arm radius $d = 0.106$ m. The equations of motion in the body frame are governed by Newton-Euler formulations:

$$\mathbf{m}\dot{\mathbf{v}} + \omega \times (\mathbf{m}\mathbf{v}) = \mathbf{F}_{\text{total}} + \mathbf{R}^T \mathbf{m}\mathbf{g}$$

$$\mathbf{I}\dot{\omega} + \omega \times (\mathbf{I}\omega) = \tau_{\text{total}}$$

## 2.2 Inner-Loop Rate PID and Inverse Mixer

To decouple attitude stabilization from high-level trajectory planning, an inner-loop rate PID controller runs at 500 Hz. The analytical inverse mixer maps total vertical thrust $T_z$ and body torques $[\tau_x, \tau_y, \tau_z]$ to individual motor forces:

$$F_0 = \frac{1}{4}T_z - \frac{1}{4d}\tau_x - \frac{1}{4d}\tau_y - \frac{1}{4k_m}\tau_z$$

$$F_1 = \frac{1}{4}T_z + \frac{1}{4d}\tau_x - \frac{1}{4d}\tau_y + \frac{1}{4k_m}\tau_z$$

$$F_2 = \frac{1}{4}T_z + \frac{1}{4d}\tau_x + \frac{1}{4d}\tau_y - \frac{1}{4k_m}\tau_z$$

$$F_3 = \frac{1}{4}T_z - \frac{1}{4d}\tau_x + \frac{1}{4d}\tau_y + \frac{1}{4k_m}\tau_z$$

# 3 Reinforcement Learning Formulation

## 3.1 Observation and Action Spaces

The observation vector $\mathbf{s}_t \in \mathbb{R}^{16}$ is defined entirely in the drone's body frame to ensure rotation invariance: - $\mathbf{e}_{p,body} \in \mathbb{R}^3$: Position error relative to target in body frame. - $\mathbf{v}_{body} \in \mathbb{R}^3$: Linear velocity in body frame. - $\mathbf{z}_{body,world} \in \mathbb{R}^3$: Projected body z-axis in world frame (tilt representation). - $\omega_{body} \in \mathbb{R}^3$: Body angular velocity $[p, q, r]$. - $\mathbf{a}_{t-1} \in \mathbb{R}^4$: Previous applied control action.

The action vector $\mathbf{a}_t \in [-1, 1]^4$ commands collective vertical thrust and desired angular rates $[\omega_x, \omega_y, \omega_z]$.

## 3.2 Jitter Mitigation and Reward Shaping

Actuator chatter is eliminated by applying an Exponential Moving Average (EMA) low-pass filter ($\alpha = 0.75$) and incorporating an action smoothness penalty into the reward function:

$$r_t = r_{\text{pos}} + r_{\text{vel}} + r_{\text{upright}} + r_{\text{alive}} - \lambda_1 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 - \lambda_2 \|\mathbf{a}_t\|^2$$

# 4 Experimental Results

## 4.1 Training Convergence

Training was executed on an Intel Core Ultra 7 155H processor with 8 parallel environments using Proximal Policy Optimization (PPO). The policy achieved convergence within 1.5 million timesteps (9 minutes 22 seconds wall-clock time), reaching an average survival length of 500/500 steps and an episode reward exceeding 940.

## 4.2 Tracking and Steady-State Performance

Under nominal evaluation across 20 unseen 3D waypoints: - **Steady-State Error**: $4.10 \pm 1.64$ cm. - **Mean Tilt Angle**: $1.58^{\circ}$ (near-zero attitude deviation). - **Action Smoothness**: $\|\Delta \mathbf{a}\| = 0.0107$, validating jitter-free command generation.

# 5 Hardware Deployment Plan on AMD Kria KR260

## 5.1 Processing System (PS) Architecture

The AMD Kria KR260 runs Ubuntu 24.04 on its Quad-Core ARM Cortex-A53 Processing System (PS). Telemetry from the simulation host is received over high-speed UDP Ethernet sockets ($<$ 0.5 ms round-trip latency), processed by ONNX Runtime, and returned to the simulator.

## 5.2 Programmable Logic (PL) FPGA Acceleration

To achieve deterministic sub-microsecond inference, the Multi-Layer Perceptron (16 -> 64 -> 64 -> 4) is synthesized into the FPGA fabric using High-Level Synthesis (HLS) with pure ReLU activation units and fixed-point arithmetic.

# 6 Conclusion

This study demonstrates the viability of training a highly precise, jitter-resistant DRL flight controller in MuJoCo and preparing it for real-time edge execution on the AMD Kria KR260. Future work involves domain randomization under extreme wind gust profiles and complete FPGA bitstream generation.